

|                                                         |                                                                                                                                                                                             |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-o &lt;outfile&gt;</code>                         | To specify the output file.                                                                                                                                                                 |
| <code>-S &lt;nn&gt;, --image_pagesize=&lt;nn&gt;</code> | Specify the AFF file page size as <code>&lt;nn&gt;</code> , which can be suffixed with <code>b</code> , <code>k</code> , <code>m</code> or <code>g</code> to reflect bytes, kilobytes, etc. |
| <code>-r</code>                                         | To produce a raw image file from an AFF image file.                                                                                                                                         |
| <code>-e &lt;ext&gt;</code>                             | To specify the extension of the output file. If the output file is not specified, the name of the output file is ' <code>&lt;input raw image name&gt;.aff</code> '                          |

Examples: To create an AFF image file from the raw image '`usb.dd`', using the LZMA compression algorithm, run the following code:

```
afconvert -z -l usb.dd
```

The output AFF file after completion of conversion will be named `usb.aff`.

To create a raw ISO image from the AFF image file '`usb.aff`', run:

```
afconvert -e iso -r usb.aff
```

After completion of conversion, the output file will be `usb.iso`.

## Afinfo

'*Afinfo*' is a command-line utility that prints information about AFF image files. Basic usage is `afinfo <options> <AFF file name>`. Some of the useful options are:

|                                 |                                                                         |
|---------------------------------|-------------------------------------------------------------------------|
| <code>-a</code>                 | Print all segments; by default, data segments are skipped.              |
| <code>-s &lt;segment&gt;</code> | Print information about <code>&lt;segment&gt;</code> ; can be repeated. |
| <code>-m</code>                 | Validate MD5 sum of entire image.                                       |
| <code>-S</code>                 | Validate SHA1 sum of entire image.                                      |
| <code>-p</code>                 | Validate the hash of each page (if present).                            |

Example: Invoking the '`afinfo`' utility for the AFF file '`/tmp/image.aff`' prints the output shown in Figure 6. Information about data segments is skipped by default.

## Afcats

'*Afcats*' is a command-line utility that prints the various contents of an AFF image file. Basic usage is `afcats <options> <AFF file name>`. Some of the useful options are:

|                             |                                                                         |
|-----------------------------|-------------------------------------------------------------------------|
| <code>-p &lt;nnn&gt;</code> | Output data of page number <code>&lt;nnn&gt;</code> .                   |
| <code>-S &lt;nnn&gt;</code> | Output data of sector <code>&lt;nnn&gt;</code> (sector size 512 bytes). |
| <code>-l</code>             | Output all segment names.                                               |
| <code>-L</code>             | Output all segment names, <code>arg</code> and segment length.          |

Example: Invoking '`afcats`' on the AFF file '`/tmp/image.aff`' with the '`-L`' option prints an output as shown in Figure 7.

## AIR Imager

AIR (Automated Image and Restore) Imager is a GUI front-end for `dd/dcfldd`, as shown in Figure 8. It has been designed to easily create forensic images or restore images on drives. It offers the following features:

- Use of either `dd` or `dcfldd`.
- Auto-detection of IDE and SCSI drives, CD-ROMs and tape drives.
- Image compression via `gzip/bzip2`.
- Image verification via MD5 or SHA1.
- Imaging over a TCP/IP network.
- Splitting images into multiple files.
- Write zeros to a drive/partition.
- Creating a large image file filled with zeros.
- Customisation of source and destination devices/files.
- Skipping of certain bytes of source and destination devices/files.

Image acquisition is one of the most important and primary tasks for any forensic investigation. Thus, it should be done carefully, and with the right set of tools—otherwise one can end up damaging the original source media. We have seen many open source tools for imaging, each one having its own advantages and disadvantages. Trying any of these will give you experience with imaging various digital media. After data acquisition and imaging comes the data recovery and forensic analysis of the forensic image files, to find digital evidence. We will learn about these two phases and the associated open source tools available in BackTrack, in the next article in this series. I hope you have enjoyed your foray into digital forensic analysis. 

## References

- [1] <http://afflib.org/downloads/README.txt>
- [2] [http://www.forensicswiki.org/wiki/Main\\_Page](http://www.forensicswiki.org/wiki/Main_Page)
- [3] <http://www.cs.harvard.edu/malan/publications/aff.pdf>
- [4] [http://www.digital-evidence.org/papers/opensrc\\_legal.pdf](http://www.digital-evidence.org/papers/opensrc_legal.pdf)

## By: Ajay Kumar Gupta

The author works with a leading wireless security company in India, and did his Masters in Electrical Engineering from IIT Bombay. For any queries, contact him at [ajaygupta.hbt@gmail.com](mailto:ajaygupta.hbt@gmail.com).